

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 15

Handling Exceptions

CHAPTER OVERVIEW

When you think of writing code as creating a component, you will realize that there are always two stakeholders involved - the provider/developer of the component and the client or the user of the component. For the c...

EXAM OBJECTIVES COVERED

- ◆ Handle exceptions using try/catch/finally, try-with-resources, and multi-catch blocks, including custom exceptions

Chapter 15 Handling Exceptions

Exam Objectives

- Handle exceptions using try/catch/finally, try-with-resources, and multi-catch blocks, including custom exceptions

15.1 Understanding exception handling

15.1.1 Java exception handling 🙌

Exceptions are for managing exceptional situations. For a file copy program, the normal course of action could be - open file A, read the contents of file A, create file B, and write the contents to file B. But what if the program is not able to open file A? What if the program is not able to create or write to file B? There could be many reasons for such failure such as a typo in the file name, lack of permission, no space on disk, or even disk failure. You don't expect these problems to occur all the time but it is reasonable to expect them to occur some times. Since we don't expect these problems to occur regularly in normal course of operation, we call them "exceptional".

In exceptional situations, you may want to give the user a feedback about the error or you may even want to take an alternative course of action. For example, you may want to let the user to input another source file name if the input file is not found or another target location if the given location is out of space. Even if you don't want to take any special action upon such situations, you should at least want your program to end gracefully instead of crashing unexpectedly at run time. This means, you should provide a path for the program to take in such situations. One way would be to check for each situation before proceeding to copy. Something like this:

```
if(checkFileAccess(file1)){
```

```

    if(checkWritePermission(targetDirectory){
        //code for normal course of action
    }else{
        System.out.println("Unable to create file2");
    }
}else{
    System.out.println("Unable to read file1");
}

```

You can see where this is going. You will end up having a lot of if-else statements. Not only is it cumbersome to code, it will be a nightmare to read and maintain later on.

There is another serious problem with the above approach. It doesn't provide way to write code for situations that you don't even know about at the time of writing the code. For example, what if the user runs this code in an environment that requires file names to follow a particular format? So, now, you have two kinds of "exceptional situations". One that you know about at the time of writing the code, and one that you don't know anything about.

Java exception mechanism is designed to help you write code that covers all possible execution paths that a program may take - 1. path for normal operation 2. paths for known exceptional situations, and 3. path for unknown exceptional situations. Here is how the above mentioned program can be written:

```

try{
    //code for normal course of action
}catch(SecurityException se){
    //code for known exceptional situation
    System.out.println("No permission!");
}
catch(Throwable t){
    //code for unknown exceptional situations
    System.out.println("Some problem in copying: "+t.getMessage());
}

```

```
t.printStackTrace();  
}
```

Observe that the perspective is reversed here. In the if/else approach, you check for each exceptional situation and proceed to copy if everything is good, while in the try/catch approach, you assume everything is good and only if you encounter a problem, you decide what to do depending on the problem. The benefits of the try/catch approach are obvious. It provides a clean separation between code for normal execution and code for exceptional situations, which makes the code easier to read and maintain. It allows an alternative path to the program to proceed even in cases where the programmer has not anticipated the problem.

Exception handling in Java is not a bag of all goodies. There has been a good amount of debate on what constitutes "exceptional situations" and what should be the right approach to handle such situations. My objective in this chapter is to teach you Java's approach to exception handling and so, I will not go into an academic discussion on whether it is good or bad as compared to other languages. However, it is a very important topic of discussion in technical interviews. I suggest you google criticism of Java's exception handling and compare it with C#'s.

15.1.2 Fundamentals of the try/catch approach

When you think of writing code as creating a component, you will realize that there are always two stakeholders involved - the provider/developer of the component and the client or the user of the component. For the client to be able to use the component, it is imperative for the provider to tell the client about "how" his component works. The how not only includes the input/output details of the component but also the information about exceptional situations, i.e., about situations the component knows may occur but does not deal with.

For example, let's say you are developing a method that copies the contents of one file

to another and that this method is to be used by a developer in another team. You must convey to the other developer that you are aware of the Input/Output issues associated with reading and writing a file but you won't do anything about them. In other words, you must convey that your method will try its best to copy the file but if there is an I/O issue, it will abandon the attempt and let her know about the failure so that she can deal with it however she wants to. This is done through the use of a **"throws"** clause in the method declaration:

```
public void copyFile(String inputPath, String outputPath) throws
IOException {
    //code to copy file
}
```

In the above code, if the `copyFile` method encounters any I/O issue while copying a file, it will simply abort the copying and throw an `IOException` to the caller. If it does not encounter any I/O issue, the method will end successfully.

On the other side of the component is the user of that component, who uses the information provided by the component provider to develop her code. The user has to decide how she wants to handle the exceptional situation. If she believes that she has the ability to "resolve" the situation, she will put the usage of that component in a **"try"** block with an associated **"catch"** block that contains code for "resolution" of the problem. When that exceptional situation actually occurs, the control goes to the catch block instead of proceeding to the next statement after the method call that threw the exception.

If the user decides that she cannot handle the exceptional situation either, she propagates the exception to the caller of her component.

For example, a method that creates a backup of a file may use the `copyFile` method internally to copy a file. If the `copyFile` method throws an `IOException`, the backup creator method may catch that exception and show a message to the user.

```
public void createBackup(String input) {
    String output = input+".backup";
```

```

try{
    copyFile(input, output);
    System.out.println("backup successful");
}catch(IOException ioe){
    System.out.println("backup failure");
}
}

```

If the `copyFile` method completes without any issue, the control will go to the `println` statement. If the `copyFile` method throws an `IOException`, the control will go to the catch block, thus, providing an opportunity to take a different course of action. Here, the code may show a failure message to the user. From the perspective of the developer of the backup program, showing the error message to the user is the resolution of the I/O problem. One can also write code to have the user specify another file or directory to create a backup.

An exception is considered "handled" when it is caught in a catch block. If the backup creator method doesn't know what to do in case the `copyFile` is not successful, then it should let the exception propagate to its caller by using a `throws` clause of its own:

```

public void createBackup(String input) throws IOException{
    String output = input+".backup";
    copyFile(input, output);
    System.out.println("backup successful");
}

```

In the above code, if the `copyFile` method completes without any issue, the control will go to the `println` statement. But if the `copyFile` method throws an `IOException`, the `createBackup` method will end immediately and the caller will receive an `IOException`. Note that this will be the same `IOException` that it receives from the `copyFile` method. This is how an exception propagates from one method to the other. There could be a long chain of method calls through which an exception bubbles up before it is handled. If an exception is not caught anywhere while it is bubbling up the call chain, it ultimately reaches the JVM code. Since the

JVM has no idea about the business logic of the program that it is executing, it "handles" the exception by killing (aka terminating) the program (actually, the JVM kills only the thread that was used to invoke the chain of method calls but since we have not talked about threading yet you can assume that a program is composed of only one thread and killing of that thread is the same as killing the program for now).

15.1.3 Pieces of the exception handling puzzle 🙌

Java exception handling mechanism comprises several moving parts. What makes this mechanism a bit complicated is that you need to put each part in its right place to make the whole thing work. So, let me introduce these parts first briefly.

The `java.lang.Throwable` object - 🙌

`Throwable` is the root class of all exceptions. It captures the details of the program and its surroundings at the time it is created. Among other things, a `Throwable` object includes the chain of the method calls that led to the exception (known as the "stack trace") and any informational message passed by the programmer as a `String` argument while creating that exception (which can be retrieved using its `getMessage()` method). This information is helpful in determining the location and the cause of the exception while debugging a program. You may have seen crazy looking output on the console containing method names upon a program crash. This output is actually the stack trace contained in the `Throwable` object. For example, if `m1()` calls `m2()`, `m2()` calls `m3()` and the code in `m3()` throws an exception, then the stack trace of that exception will contain the names of the methods `m3`, `m2`, and `m1`, in that order.

`Throwable` has two subclasses - `java.lang.Error` and `java.lang.Exception`, and a huge number of grand child classes. Each class is meant for a specific situation. For example, a `java.lang.NullPointerException` is thrown when the code tries to access a null reference or an `java.lang.ArrayIndexOutOfBoundsException` is thrown when you try to access an array beyond its size.

What is called "**exception**" (i.e. exception with a lower case 'e') in common parlance, is in reality is an object of class `java.lang.Throwable` or one of its subclasses. Although you will rarely use the `Throwable` class itself directly in your code, it contains most of the functionality related to exceptions and so, you should be aware of the constructors and important methods of this class.

Throwable constructors

Throwable has the following are three public constructors:

1. **`Throwable()`** : Constructs a new throwable with null as its detail message.
2. **`Throwable(String message)`** : Constructs a new throwable with the specified detail message.
3. **`Throwable(String message, Throwable cause)`** : Constructs a new throwable with the specified detail message and the "cause". The cause is another throwable object that the developer believes is the real cause of the exception. This is useful if you want to catch a standard exception such as `NullPointerException` and then throw a new business exception with the original exception as the real cause.

Throwable methods

You may see the usage of the following methods of Throwable on the exam:

1. **`String toString()`** : Returns the string representation of the throwable in `<exception class name>: <message>` format.
2. **`String getMessage()`** : Returns the detail message string of this throwable.
3. **`void printStackTrace()`** : Prints the stack trace in the error stream.
4. **`Throwable getCause()`** : Returns the cause that was passed while constructing the throwable.
5. **`void addSuppressed(Throwable exception)/Throwable[] getSuppressed()`** : Add another throwable to its array of suppressed exceptions and get the array of suppressed exception.

One subclass of `java.lang.Exception` that is particularly important is `java.lang.RuntimeException`. `RuntimeException` and its subclasses belong

to a category of exception classes called **"unchecked exceptions"**. You will see their significance soon.

The `throw` statement 🙌

The `throw` statement is used by a programmer to "throw an exception" or "raise an exception" explicitly. A programmer may decide to throw an exception upon encountering a condition that makes continuing the execution of the code futile. For example, if, while executing a method, you find that the value of a required parameter is invalid, you may throw an `IllegalArgumentException` using a `throw` statement like this:

```
public double computeSimpleInterest(double p, double r, double t){
    if( t<0) {
        IllegalArgumentException iae = new
IllegalArgumentException("time is less than 0");
        throw iae;
    }
    //other code
}
```

Usually, an exception object is created only to be "thrown" and so, there is no need to store its reference in a variable. That is why it is often created and thrown in the same statement as shown below:

```
public double computeSimpleInterest(double p, double r, double t){
    if( t<0) throw new IllegalArgumentException("time is less than 0");
    //other code
}
```

Throwing an exception implies that the code has encountered an unexpected situation with which it does not want to deal. The code shown above, for example, expects the

time argument to be greater than zero. For this code, time being less than zero is an unexpected situation. It does not want to deal with this situation (probably because the programmer is not sure what to do in this case) and so it throws an exception in such a situation. This also means that this method passes on the responsibility of determining what should be done in case time is less than zero to the user of this method.

Throwing an exception abandons the current flow of execution and the rest of the statements in that flow are not executed. The control leaves the current point of execution and goes to the point where that exception is "handled". For example, throwing an exception from a `for` loop will cause the loop to break:

```
for(int i=0; i<10; i++){
    if(i == 5) throw new Exception();
    System.out.println(i);
}
System.out.println("after loop"); //will not be executed
```

In the above loop, an exception is thrown in the 6th iteration and so the rest of the iterations will not be executed. Even the code following the `for` loop will be skipped. Where exactly the control will go after executing the throw statement depends on where this exception is handled. This will be clear soon.

Note that only an instance of `Throwable` (or of its subclasses) can be thrown using the `throw` statement. You cannot do something like `throw new Object();` or `throw "bad situation";`

Explicitly throwing an exception using the throw statement is not the only way an exception can be thrown. JVM may also decide to throw an exception if the code tries to do some bad thing like calling a method on a null reference. For example:

```
public void printLength(String str){
    System.out.println(str.length());
}
```

```
}
```

If you pass `null` to the above method, the JVM will create a new instance of `NullPointerException` and throw that instance when it tries to execute `str.length()`.

The `throws` clause 🙋

Java requires that you list the exceptions that a method might throw in the `throws` clause of that method. This ties back to Java's design goal of letting the user know of the complete behavior of a method. It wants to make sure that if the method encounters an exceptional situation, then the method either deals with that situation itself or lets the caller know about that situation. The `throws` clause is used for this purpose. It conveys to the user of a method that this method may throw the exception mentioned in the `throws` clause. For example:

```
public double computeSimpleInterest(double p, double r, double t)
throws Exception{
    if( t<0) throw new Exception("time is less than 0");
    //other code
}
```

Now, anyone who uses the above method knows that this method may throw an exception instead of returning the interest. This helps the user write appropriate code to deal with the exceptional situation if it arises.

The `try` statement 🙋

A **try statement** gives the programmer an opportunity to recover from and/or salvage an exceptional situation that may arise while executing a block of code. A try statement consists of a **try block**, zero or more **catch blocks**, and an optional **finally block**. Its syntax is as follows:

```
try {  
    //code that might throw exceptions  
} catch (ExceptionClass1 e1) {  
    //code to execute if code in try throws exception 1  
}  
catch (ExceptionClass2 e2) {  
    //code to execute if code in try throws exception 2  
}  
catch (ExceptionClassN en) {  
    //code to execute if code in try throws exception N  
}  
finally {  
    //code to execute after the try block and catch block finish  
    execution  
}
```

Note that curly braces for the `try`, `catch`, and `finally` blocks are required even if there is a single statement in these blocks (compare that to `if`, `while`, `do/while` and `for` blocks where curly braces are not required if there is only one statement in the block). Furthermore, a **try block** must follow with at least one **catch block** or the **finally block**. A try block that follows with neither a catch block nor a finally block will not compile.

The `try` statement is the counterpart of the `throw` statement because putting a piece of code within a try block signifies that the programmer wants to do something in case that piece of code throws an exception. In other words, a `try` block lets the programmer deal with an exceptional situation as opposed to the `throw` statement, which lets the programmer avoid dealing with it.

While a `try` block contains code for normal operation of the program, a **catch block** is the location where the programmer tries to recover from an exceptional situation that arises in the `try` block. This is also called "handling" the exception. If the code in the `try` block throws an exception, the normal flow of execution in that

`try` block is aborted (i.e. no further code in the `try` block is executed) and the control goes to the `catch` block. Here, the programmer can take alternate approach to finish the processing of the method. For example, the programmer may decide to just show a popup message to the user about the exception and move on to the next statement after the `try` statement.

A catch block is associated with a **catch clause**, which specifies the exception that the catch block is meant to handle. For example, a `catch` block with the catch clause as `catch(IllegalArgumentException e)` is meant to handle `IllegalArgumentException` and its subclasses. Thus, this `catch` block will be executed only if the code in the `try` block throws an `IllegalArgumentException` or its subclass exception. You can specify any valid exception class (including `java.lang.Throwable`) in the `catch` clause.

A **finally block** is the location where the programmer tries to salvage the situation or control the damage so to say, without attempting to recover from an exception. For example, a program that tries to copy a file may want to close any open files irrespective of whether the copy operation is successful or not. The programmer can do this in the finally block. You may think of a finally block as the step where a car mechanic reassembles the parts back irrespective of whether he was able to fix the car or not!

Here is a method that calls the `computeSimpleInterest` method shown above within a `try` statement:

```
public void doInterest(){
    try{
        double interest = computeSimpleInterest(1000.0, 10.0, -1);
        System.out.println("Computed interest "+interest);
    }catch(IllegalArgumentException e){
        System.out.println("Problem in computing interest:"+e.getMessage());
        e.printStackTrace();
    }
}
```

```
    }finally{  
        System.out.println("all done");  
    }  
}
```

In the above code, the call to `computeSimpleInterest` throws an `IllegalArgumentException` because `t` is negative. Thus, the `println` statement after the method call is not executed. The exception is caught in the `catch` block because its catch clause, i.e., `catch(IllegalArgumentException )` matches with the exception that is thrown by the `try` block and the control goes to the first statement in this catch block. It prints the exception's message and the stack trace on the console. Finally, the control goes to the code in the finally block, where it prints `"all done"`. If you omit the catch block, the control will go directly to the finally block after the invocation of `computeSimpleInterest` method. After the execution of the code in the finally block, the caller of `doInterest` method will receive the same `IllegalArgumentException`.

Note that a finally block, if present, always executes irrespective of what happens in the `try` block or the catch block. Even if the `try` block throws an exception and there is no `catch` block to catch that exception, the JVM will execute the finally block. It will throw the exception to the caller only after the `finally` block finishes. The only case where the `finally` block is not executed is if the code in the `try` or the catch block forces the JVM to shut down using a call to `System.exit()` method.

The following is a complete program that illustrates how an exception alters the normal program flow:

```
public class TestClass{  
  
    public static void main(String[] args){  
        TestClass tc = new TestClass();  
        tc.doInterest();  
    }  
}
```

```

public double computeSimpleInterest(double p, double r, double t)
{
    if( t<0) throw new IllegalArgumentException("time is less than
0");
    return p*r*t/100;
}

public void doInterest(){
    try{
        double interest = computeSimpleInterest(1000.0, 10.0, -1);
        System.out.println("Computed interest "+interest);
    }catch(IllegalArgumentException iae){
        System.out.println("Problem in computing
interest:"+iae.getMessage());
        iae.printStackTrace();
    }finally{
        System.out.println("all done");
    }
}
}

```

It generates the following output on the console :

```

Problem in computing interest:time is less than 0
java.lang.IllegalArgumentException: time is less than 0
    at TestClass.computeSimpleInterest(TestClass.java:8)
    at TestClass.doInterest(TestClass.java:14)
    at TestClass.main(TestClass.java:4)
all done

```

You should observe the following points in the above code:

1. The `return` statement in `computeSimpleInterest` is not executed because the previous statement throws an exception.
2. The `println` statement in the try block of `doInterest` is not executed because the call to `computeSimpleInterest` ends with an exception instead of a return value. Control goes to the `catch` block directly after the method call.
3. The `catch` block prints the details captured in the exception object. It shows the sequence of the method invocations in reverse order from the point where the `IllegalArgumentException` object was created. You should try removing the catch block and see the output.
4. The call to `iae.getMessage()` returns the message string that was passed while creating this exception object. In the exam, you may also see an implicit invocation of the `toString()` method of an exception object. This method returns a `String` containing the exception class name and the exception message. For example, `System.out.println(iae);` will print `java.lang.IllegalArgumentException: time is less than 0`.
5. Once the `catch` block is finished, the control goes to the `finally` block.
6. The `doInterest` method returns after the execution of the `finally` block.
7. There is no throws clause in `computeSimpleInterest` method even though it throws an exception. The reason will be clear in the next section where I talk about checked and unchecked exceptions.

15.1.4 Returning a value from try, catch, and finally blocks 🙌

A `return` statement returns a value and ends the method execution immediately ignoring the rest of the method code, while a finally block prevents the termination of a method before the code in the `finally` block is executed. Thus, the `return` statement and the `finally` block impose contradictory requirements on the flow of execution. This is not a problem when a `return` statement is present in the `try` or the `catch` blocks. The JVM simply holds the value that the `return` statement wants to return, executes the `finally` block, and then returns the value to the caller. For

example, the following code prints In finally **before** the value 1 or 2 is returned to main:

```
public static int getValue() throws Exception{
    try{ if(Math.random()>0.5) throw new Exception();
        return 1;    }
    catch(Exception e){ return 2; }
    finally{ System.out.println("In finally"); }
}
public static void main(String[] args) throws Exception{
    System.out.println("value = "+getValue());
}
```

Things get interesting if the finally block too has a return statement. Can you guess what will happen if I update the finally block in the above code to: finally{ System.out.println("In finally"); return 3; }. The getValue method is about to return a value 1 or 2 but the the finally block provides a return value of 3. Which value should the method return to the caller? Java resolves this dilemma by mandating that the value returned by the finally block supersedes any value returned by the try or the catch blocks. The value returned by the try or the catch blocks are discarded.

Similarly, a return statement from a finally block can also cause an exception thrown from the the try or the catch blocks to be gobbled. For example, the exception thrown from the try block in the following method will be discarded and the method will always return 3 to the caller:

```
public static int getValue(){ //throws clause is not needed
    try{ throw new Exception(); }
    finally{ System.out.println("In finally"); return 3; }
}
```

The compiler realizes that since there is a return statement in finally, the method will never end up throwing an exception to the caller and so, the throws clause is not

required anymore.

Just like the return statement, throwing an exception from the finally block also makes the JVM discard the exception thrown from the try or catch blocks. For example:

```
public static int getValue() throws Exception{
    try{ if(true) throw new Exception("Exception from try"); }
    catch(Exception e){ throw new Exception("Exception from catch", e); }
    finally{ throw new Exception("Exception from finally"); }
}
```

The exception thrown from the try block is caught by the catch block, which throws a new exception with the original exception as its cause. But this exception will be lost and the caller of this method will always receive an exception containing the message "Exception from finally" .

15.2 Understanding checked exceptions, unchecked exceptions, and errors

15.2.1 Checked and Unchecked exceptions

As discussed earlier, exceptions thrown by a method are part of the contract between the method and the user of that method. If the user is not aware of the exceptions that a method might throw, they will be blindsided during run time because their code would not be prepared to handle that exception. The `throws` clause of a method is meant to list all exceptions that the method might throw. If an exception is listed in the throws clause, the user of the method will know that they need to somehow handle that situation.

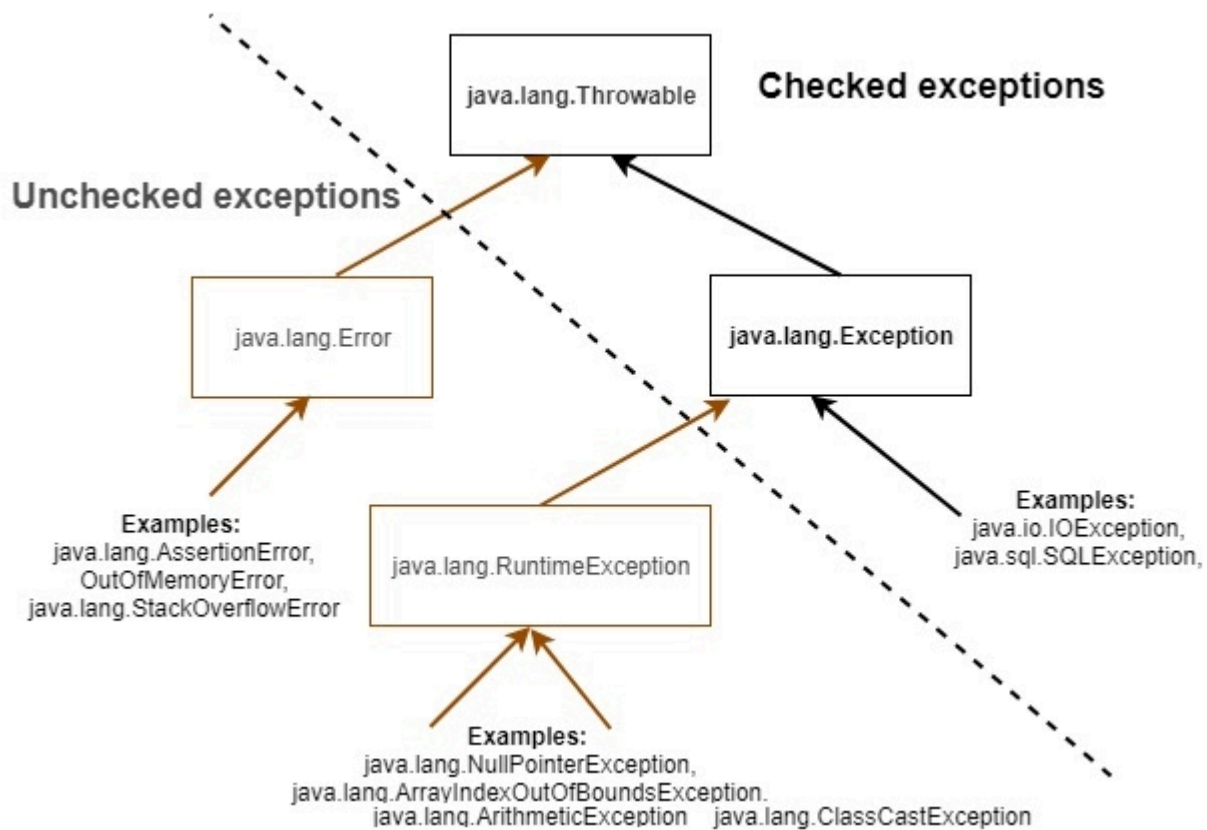
But what if a developer forgets to list an exception in the throws clause of a method? In that case, we are back to the same problem of blindsiding the user of that method at run time. This is where the compiler plays an important role. While compiling a method, the compiler checks for the possibility of any exception that might get thrown by the method to the caller. If that exception is not listed in the throws clause of that method, it refuses to compile the method.

This sounds like a good solution, but the problem here is that the compiler does not check for all kinds of exceptions thrown by a piece of code. It checks for only a certain kind of exceptions called "**checked exceptions**" and forces you to list the only those exceptions that belong to this category of exceptions in the throws clause.

Exceptions that do not belong to the category of checked exceptions are called "**unchecked exceptions**". They are called unchecked because the compiler doesn't care whether a piece of code throws such an exception or not. Listing unchecked exceptions in the throws clause is optional.

Finding out whether an exception is a checked exception or not is easy. Java language designers have postulated that any exception that extends `java.lang.Throwable` but does not extend `java.lang.RuntimeException` or `java.lang.Error` is a checked exception. The rest (i.e. `java.lang.Error`, `java.lang.RuntimeException`, and their subclasses) are unchecked.

The following figure illustrates this grouping of exceptions.



Note that Java has a deep rooted exception class hierarchy, which means there are several classes, subclasses, and subclasses of subclasses in the tree of exceptions. So just because an exception is a `RuntimeException`, does not mean that that exception directly extends `RuntimeException`. It could even be a grand child of `RuntimeException`. For example, `ArrayIndexOutOfBoundsException` actually extends `IndexOutOfBoundsException`, which in turn extends `RuntimeException`. Similarly, just because an exception is a checked exception does not mean that it directly extends `Exception`. It may extend any subclass of `Exception` (except `RuntimeException`, of course).

Rationale behind checked and unchecked exceptions 🙌

Recall our discussion on exceptional situations where I talked about two kinds of exceptional situations - ones that a developer knows about and ones that a developer doesn't expect to occur at all. While a developer may want to provide an alternate path

of execution in case of a situation that is known to occur but there is no point in providing an alternate path of execution for a situation that is never expected to occur.

For example, if a piece of code tries to write to a file, the developer may want to take a different approach if he is not able to write to the file system. But if the data array that it is trying to write is null, there is nothing much he can do. Such unexpected situations occur mostly due to badly written code. In other words, if a code gets itself into an unexpected situation, it is most likely because of a programming error, i.e., a bug in the code. Such issues should be fixed during development itself. But if they do occur in production, they should rather be handled at a much higher level than at the component level.

Unchecked exceptions are for such **unexpected** situations. Java language designers believe that unchecked exceptions need not be declared in a method declaration because there is nothing to gain by forcing the caller to catch them. Only checked exceptions need to be declared because the caller of the method may have a plan to recover from them. There are two kinds of unchecked exceptions - exceptions that extend `java.lang.RuntimeException` (aka **runtime exceptions**) and exceptions that extend `java.lang.Error` (aka **errors**).

Runtime exceptions and errors

Characterizing a situation as expected or unexpected is a design decision that depends on the business purpose of the method. One method may expect to receive a null argument and may work well if it gets a null argument, while another may not expect a null argument and may end up throwing a `NullPointerException` when it tries to access that null. In the second case, passing a null to that method would be considered a bug in the code, which must be identified and fixed during testing. Such exceptions that signify the presence of a bug in the code are categorized as **runtime exceptions**.

Here, the word runtime in `RuntimeException` does not imply that only exceptions that extend `RuntimeException` can be thrown at run time. All exceptions are thrown only when the program is executed, i.e., at run time. It refers to the fact that the developer

comes to know of the occurrence of the situation that results in a `RuntimeException` only when the program is run, i.e., during run time. Had the programmer anticipated the occurrence of that situation during compile time, he would have fixed the code, and in which case the exception would not have been thrown upon running the program. For the same reason, runtime exceptions are usually not thrown explicitly using the `throw` statement. Indeed, why would you write the code to throw an exception if you don't even expect that situation to occur. The JVM throws runtime exceptions on its own when it encounters an unexpected situation. For example, if you try to access a null reference, the JVM will throw a `NullPointerException`, or if you try to access an array beyond its size, the JVM will throw an `ArrayIndexOutOfBoundsException`. It is possible to recover from runtime exceptions but ideally, since they indicate bugs in the code, you should not attempt to catch them and recover from them. A well written program should not cause the JVM to throw runtime exceptions.

The case of **errors** is similar to **runtime exceptions**. The difference is that errors are reserved for situations where the operation of the JVM itself is in jeopardy. For example, a badly written code may consume so much memory that there is no free memory left. Once that happens, the JVM may end up throwing `OutOfMemoryError`. Similarly, a bad recursion may cause `StackOverflowError` from which no recovery is possible. Errors signify serious issues in the interaction between the code and the JVM and are thrown exclusively by the JVM. It is never a good idea to throw them explicitly or to try to recover from them because the code will likely not work as expected anyway once the JVM starts throwing Errors.

Although the exam does not focus on the reason for categorizing exception between checked and unchecked exceptions, it is actually a very important topic to understand for a professional developer. You should also compare Java's exception handling with C#'s.

Identifying exceptions as checked or unchecked 🙌

Java follows a convention in naming exception classes. This convention is sometimes

helpful in determining the kind of exception you are dealing with. The name of any class that extends `Error` ends with **Error** and the name of any class that extends `Exception` ends with **Exception**. For example, `OutOfMemoryError` and `StackOverflowError` are Errors while `IOException`, `SecurityException`, `IndexOutOfBoundsException` are Exceptions.

However, there is no way to distinguish between unchecked exceptions that extend `RuntimeException` and checked exceptions just by looking at their names. It is therefore, important to memorize the names of a few important runtime exception classes, namely - `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `ClassCastException`, `IllegalArgumentException`, and `SecurityException`.

15.2.2 Commonly used exception classes [👉](#)

The Java standard library contains a large number of exception classes but the exam expects you to know about only a few of them, which I will cover now. These exception classes are important because you will encounter them quite often while working with Java.

You may have read about the classification of exceptions based on whether they are thrown by the programmer or by the JVM. It is true that there are a few exceptions that are thrown by the JVM on its own. However, there is no exception that is thrown exclusively by the JVM. In fact, many methods of the Java standard library classes throw the same exceptions explicitly using the `throw` statement.

I have omitted the package name from the classes below for brevity but note that all of the following exception classes belong to the `java.lang` package.

Common Exceptions that are usually thrown by the JVM [👉](#)

1. `ArithmeticException` extends `RuntimeException`

1. The JVM throws this exception when you try to divide a number by zero.

Example :

```
public class X {  
    static int k = 0;  
    public static void main(String[] args){  
        k = 10/0; //ArithmeticException  
    }  
}
```

2. `ClassCastException` extends `RuntimeException`

The JVM throws this exception when you try to cast a reference variable to a type that fails the IS-A test. We have already gone through this in chapter 12.

Example :

```
Object s = "asdf";  
  
StringBuffer sb = (StringBuffer) s; //ClassCastException because s  
    refers to a String and not a StringBuffer.
```

3. `IndexOutOfBoundsException` extends `RuntimeException`

This exception is a common superclass of exceptions that are thrown where an invalid index is used to access a value that supports indexed access.

For example, the JVM throws its subclass `ArrayIndexOutOfBoundsException` when you attempt to access an array with an invalid index value such as a negative value or a value that is greater than the length (minus one, of course) of the array. Methods of `String` class throw another of its subclass `StringIndexOutOfBoundsException` when you try to access a character at an invalid index.

Example :

```
int[] ia = new int[]{ 1, 2, 3}; // ia is of length 3.
```


3.

```
System.out.println(ia[-1]); //ArrayIndexOutOfBoundsException
System.out.println(ia[3]); //ArrayIndexOutOfBoundsException

System.out.println("0123".charAt(4)); //StringIndexOutOfBoundsException
```

4. `NullPointerException` extends `RuntimeException`

The JVM throws this exception when you attempt to call a method or access a field using a reference variable that is pointing to null.

Example :

```
String s = null;
System.out.println(s.length()); //NullPointerException because s is null.
```

Common Errors usually thrown by the JVM 🙌

1. `ExceptionInInitializerError` extends `Error`

The JVM throws this Error when any exception is thrown while initializing a static variable or a static block.

Example :

```
public class X {
    static int k = 0;
    static{
        k = 10/0; //throws ArithmeticException but it is wrapped
        into an ExceptionInInitializationError
    }
}
```

2. `OutOfMemoryError` extends `Error`

The JVM throws this Error when it runs out of memory. This usually happens when a program creates too many objects.

2. Example:

```
public static void main (String[] args) {
    StringBuilder sb = new StringBuilder("a long string");
    for(int i=0; i<Integer.MAX_VALUE; i++){
        sb.append(sb.toString());
    }
}
```

3. `StackOverflowError` extends `Error`

The JVM throws this Error when the thread executing the method runs out of stack space. This usually happens when a method calls itself recursively and there is no boundary condition to stop the recursion.

Example :

```
public void m1(int k){
    m1(k++); //StackOverflowError
}
```

Exceptions thrown by Application Programmer 🙌

All instances of `Exception` and its subclasses are generally thrown by the application programmer using the `throw` statement. In some cases, application programmer may throw `RuntimeException` s and `Error` s as well.

1. `IllegalArgumentException` extends `RuntimeException`

This exception is thrown when a method receives an argument that the programmer has determined is not legal.

Example:

```
public void processData(byte[] data, int datatype)
{
    if(datatype != 1 && datatype != 2)
```

1.

```
        throw new IllegalArgumentException("Invalid datatype  
        "+datatype);  
        else System.out.println("Data Processed.");  
    }
```

2. `NumberFormatException` extends `IllegalArgumentException`

This exception is thrown when a method that converts a `String` to a number receives a `String` that it cannot convert.

Example:

```
int i = Integer.parseInt("asdf");//a NumberFormatException will be  
thrown by the parseInt method
```

3. `SecurityException` extends `RuntimeException`

This exception is thrown if the Security Manager refuses to permit the requested operation due to restrictions placed by the JVM. For example, when a Java program runs in a sandbox (such as an applet) and tries to use prohibited APIs such as File I/O, the security manager throws this exception. Since this exception is explicitly thrown using the new keyword by a security manager class, it can be considered to be thrown by the application programmer.

4. `java.io.IOException` extends `Exception`

This exception is the root class of all exceptions thrown by classes that perform any kind of I/O operations. For example, a method that tries to open a file may throw `java.io.FileNotFoundException`, which is a subclass of `IOException`.

5. `NoClassDefFoundError` extends `Error`

Although it is an Error but it is not thrown by the JVM. It is thrown by a class loader (which is just another class in Java standard library) when it is not able to find the definition of a class that it is trying to load. Beginners get this error often while trying to run their program. For example, if your class has a package statement but you have not put the class file in its proper directory structure, the system class loader will not be able to find that class when you try to run it and

5. will throw this error.

15.2.3 Creating custom exception classes 📌

Given the different types of exception classes in the JDK (and in application frameworks such as Spring and Hibernate), there is usually no need to create custom exception classes to represent programming related issues. However, custom exception classes may be needed for representing exceptional situations that are unique to an application. For example, an accounting application could define a `LowBalanceException` for a situation where the amount being withdrawn from an account is more than what the account balance or withdrawal limit. The method that encounters such as situation may decide to abort further processing by throwing an instance of a `LowBalanceException`. For example:

```
public class LowBalanceException extends Exception{
    String acctNo; double currBal; double amtRequested;
    public LowBalanceException(String acctNo, double currBal, double
amtRequested){
        super("Account "+acctNo+" does not have enough balance.");
        //init fields
    }
}
... in another place
try{
    ... logic for money transfer
}catch(LowBalanceException lbe){
    log(lbe);
    //forward the user to another page
}
```

Creating a custom exception class not only allows extra information about the

exceptional situation to be included in an exception but also helps with the control flow. Typically, application frameworks such as Spring allow you to declaratively specify (through config files or annotations) what action to take when a particular exception is encountered.

A custom exception class may extend any of the existing exception classes. But remember that all the rules about checked and unchecked exceptions apply to custom exception classes as well. Thus, the `LowBalanceException` defined above is a checked exception.

15.3 Creating and invoking methods that throw an exception

15.3.1 Creating a method that throws an exception

Now that you know about the throws clause and the types of exceptions, let us look at the rules for creating a method that throws an exception. Actually, there is just one rule - If there is a possibility of a **checked exception** getting thrown out of a method, then that exception or its superclass exception must be declared in the **throws clause** of the method. The following are examples of a few valid methods that illustrate this rule:

```
1. void process(int x) throws Exception{
    if(x == 2) throw new Exception(); //throws Exception only if
    x==2
    else return;
}
```

It doesn't matter whether the code throws an exception every time it runs or only some times. If there is a path of execution in which an exception will be throw, the method must list that exception in the throws clause.

2.

```
void process() { //no throws clause necessary
    if(someCondition) throw new RuntimeException();
    else throw new Error();
}
```

`RuntimeException` and `Error` are **unchecked exceptions** and are therefore, exempt from being declared in the throws clause. Declaring them in the throws clause is valid though.

3.

```
void process() throws Exception{
    if(someCondition) throw new java.io.IOException(); //throwing a sub-exception
    else return;
}
```

It is ok to declare a superclass exception in the throws clause and throw a subclass exception in the method. But the reverse is not acceptable:

```
void process() throws java.io.IOException{ //will not compile
    if(someCondition) throw new Exception(); //throwing a super-exception
    else return;
}
```

Remember that a throws clause is a commitment that you give to the user of this method that this method may throw only those exceptions that are listed in the throws clause. If you commit that you may only throw an `IOException` but then throw an `Exception` instead, then the caller will have a problem because the caller is only prepared to handle an `IOException` and not an `Exception`. Throwing an `Exception` will break the caller's code.

Declaring a broader exception (i.e. a superclass exception) and throwing a

3. narrower exception (i.e. a subclass exception) is ok because if the caller is already prepared to handle the broader exception it can handle the narrower one without changing any code. For example, an `IOException` is an `Exception` and by throwing `IOException`, you are not breaking your commitment if you have committed that you may throw an `Exception`.

4.

```
void process() throws java.io.IOException, java.sql.SQLException //can
throw a common superclass exception as well.
{
    if(someCondition) throw new java.io.IOException();
    else throw new java.sql.SQLException();
}
```

If a method throws multiple exceptions, you can either list them individually or list a common superclass. The logic behind this is the same as before. As long as the method is true to what it has committed in its declaration, it is good!

5.

```
void process() throws Exception, java.io.IOException{//specifying
IOException is redundant because IOException is a subclass of
Exception
    if(someCondition) throw new java.io.IOException();
    else throw new java.sql.SQLException();
}
```

It is ok for a method to declare a superclass exception as well as a subclass exception even though adding the subclass exception to the list when a superclass is present is redundant.

6.

```
void process() throws Exception{ //declaring Exception in the
throws clause even though it is not thrown by the method body
    System.out.println("hello");
}
```

6. }

A method can declare any exception in its throws clause irrespective of whether that method actually throws that exception or not. It is sometimes useful to "future-proof" a method by declaring Exception in the throws clause if you believe that the method's implementation may change later. By declaring Exception in the throws clause, the users of the method will not have to change their code if the method actually starts throwing Exception or any of Exception subclasses later.

```
7. void process1() {  
    try{  
        if(someCondition) throw new Exception(); //will be caught by  
the catch block  
        else return;  
    }catch(Exception e){  
  
    }  
}
```

The requirement to list an exception in the throws clause is applicable only when an exception is thrown out of the method to the caller. If the code inside a method throws an exception but that exception is caught within the method itself, there is no need to declare it in the throws clause of the method.

Remember that an exception can only be caught by a catch block and not by a finally block. Therefore, the throws clause is necessary in the following method:

```
void process2() throws Exception{  
    try{  
        if(someCondition) throw new Exception();  
        else return;  
    }finally {  
        System.out.println("in finally"); //will be executed  
    }  
}
```


7.

```
    but the exception is not caught here
    }
}
```

8.

```
void process() throws Throwable{
    if(true) throw new Exception();
    else return;
}
```

As you know, `Throwable` is the root of all exception classes, and therefore, if you declare `Throwable` in the throws clause, you can throw any exception in the method.

Although not important for the exam, deciding which exception to throw and which to declare is an important matter.

Declaring exceptions 🙌

As shown in point number 8 above, declaring a broad exception class in the throws clause is an easy way to get rid of any compilation errors with the method if the method throws different kinds of exceptions from different parts of its code. However, this is considered a bad practice because this burdens the user of the method with dealing with a broad range of exceptions.

On the other hand, listing specific exception classes individually restricts the future modifiability of a method because throwing new exceptions later will break other people's code.

It is recommended to be balanced in your approach towards listing exceptions in the throws clause. You should try to be only as specific as is possible without compromising the modifiability of the method. For example, I/O related methods may encounter different kinds of issues while reading a file. It may not be

possible to identify all such issues while writing the method. New issues may be discovered later and you may have to modify your method to accommodate those. Therefore, it is better to declare a common superclass `IOException` in the throws clause instead of individual subclasses such as `FileNotFoundException` or `EOFException`.

Throwing exceptions [👉](#)

You should always throw the most specific exception possible. For example, if you don't find the file while trying to open it, you should throw a `FileNotFoundException` instead of an `IOException` or `Exception`. By throwing the most specific exception, you give more information to the caller about the problem. This helps the caller in determining the most suitable resolution of the problem.

15.3.2 Throwing exceptions from initializers and constructors [👉](#)

Throwing exceptions from static initializers [👉](#)

The JVM executes static initializers automatically when it loads a class. Although the JVM loads a class due to some action taken by the application code such as creating an object of the class or invoking a static method of the class, there is no direct invocation of a static initializer from the application. Therefore, if a static initializer ends up throwing an exception, there is no way for the application to handle that exception and to recover from it. For this reason, a static initializer is not allowed to throw any checked exception. If the compiler sees a possibility of a checked exception getting thrown out of a static initializer, it will generate an error. For example, the following code will not compile:

```
public class TestClass{
    static int i = 5;
    static{
        if(i == 0) throw new Exception();
    }
}
```

The error generated by the compiler message says, "Error: unreported exception java.lang.Exception; must be caught or declared to be thrown", which is a bit misleading because there is no where to declare the exception!

Throwing exceptions from instance initializers 🙄

Instance initializers are always executed when the application code tries to create an instance of the class. Thus, it is possible for the application code to catch an exception thrown by an instance initializer and for the same reason, instance initializers are allowed to throw checked exceptions. However, an instance initializer has the same problem as a static initializer - there is no way to specify a throws clause for an instance initializer. Recall that an instance initializer is executed no matter which constructor of the class is invoked. In that sense, an instance initializer is kind of a part of each constructor of the class. Therefore, an exception thrown from an instance initializer can be thought of as an exception thrown by every constructor of the class. Thus, if we declare an exception thrown from an instance initializer in the throws clause of each constructor of the class, we should be good. That is exactly what Java mandates. Here is an example:

```
public class TestClass{
    int i = 5;
```

```

{
    if(i == 0) throw new Exception();
}

TestClass() throws Exception{
}

TestClass(int x) throws Exception{
}
}

```

In the above code, since the instance initializer of `TestClass` throws `Exception`, each of its two constructors must declare `Exception` in their throws clauses. Now, can you guess whether the following code will compile or not?

```

public class TestClass{
    int i = 5;

    {
        if(i == 0) throw new Exception();
    }

}

```

Observe that `TestClass` does not define any constructor explicitly. Therefore, the compiler will provide the default no-args constructor for this class. However, the default constructor does not have any throws clause. Therefore, the exception thrown by the instance initializer is not getting declared by that constructor. Hence, the above code will not compile.

Throwing exceptions from constructors [👉](#)

Just like a method, a constructor is allowed to throw any exception as long as it declares that exception in its throws clause. However, there is one important difference between a method and a constructor. Recall that the first line of every constructor is always a call to a constructor of its super class or to another constructor

of the same class. Thus, if a constructor decides to throw an exception, it has an impact on the subclass because that exception will be propagated to the subclass constructor as well. Therefore, if a subclass constructor invokes a superclass constructor that throws an exception, that subclass constructor must also declare that exception in its throws clause. Here is an example:

```
class Fruit{
    Fruit() throws Exception{
        if(Math.random()>0.5) throws Exception; //throws an exception randomly
    }

    Fruit(int calories){
    }

}
```

The `Fruit` class above has two constructors - one with a throws clause and one without. Now let's see some subclasses of `Fruit`:

```
class Apple extends Fruit{ //will NOT compile.
}
```

Notice that since `Apple` doesn't define any constructor explicitly, the compiler will provide the default no-args constructor to this class. Furthermore, the compiler will insert a call to `super();` in the default constructor. This default constructor will cause the `Apple` class to fail compilation because it has neither a throws clause nor a wrapping try/catch block that could catch the exception thrown by the call to `super();`, which is nothing but a call to `Fruit`'s no-args constructor. To make it compile, you must provide the no-args constructor with an appropriate throws clause yourself:

```
class Apple extends Fruit{
    Apple() throws Exception{
```

```

    }

    Apple(int calories) { //no throws clause needed
        super(calories);
    }
}

```

Observe that the int constructor of `Apple` does not require any throws clause because it invokes `Fruit`'s int constructor, which does not throw any exceptions. You could also change `Apple`'s no-args constructor to invoke `Apple`'s int constructor using `this(100);` to avoid having a throws clause:

```

class Apple extends Fruit{
    Apple() {
        this(100);
    }

    Apple(int calories) { //no throws clause needed
        super(calories);
    }
}

```

A subclass constructor is free to throw any new exception along with the exceptions listed in the throws clause of a super class constructor.

Note that it is not possible for a subclass constructor to catch an exception thrown by the superclass constructor because to do that you would have to put the call to `super( ... )` in a try block, in which case the call to `super( ... )` will not be at the first line of the constructor!

15.3.3 Invoking a method that throws an exception 🙌

The rules for invoking a method that throws an exception are similar to the ones for creating a method that throws an exception. As far as the compiler is concerned, there is no difference between using a `throw` statement to throw an exception and invoking a method that throws an exception to throw an exception. In both the cases, the compiler forces you to either declare the exception in your `throws` clause or handle the exception using a `try/catch` block. Of course, as discussed before, the compiler is only concerned with **checked exceptions**. The following example illustrates this point:

```
public class TestClass{
    public static void message() throws Exception{
        if(true) throw new Exception();
        else return;
    }
    public static void main(String[] args) throws Exception {
        message();
    }
}
```

In the above code, the method `message()` throws an `Exception` using the `throw` statement. Since `Exception` is a checked exception, the compiler forces it to be declared in the `throws` clause. On the other hand, instead of throwing an exception explicitly using the `throw` statement, `main` invokes `message`. But the compiler knows that invoking `message` may result in an `Exception` being thrown (because it is mentioned in the `throws` clause of `message`) and so the compiler forces `main` to declare `Exception` in its `throws` clause as well. The other option for `main` is, of course, to handle the exception itself:

```
public static void main(String[] args) { //no throws clause
necessary
    try{
        message();
    }catch(Exception e){
        e.printStackTrace();
    }
}
```

```
}  
}
```

It is ok to use a catch block with a broader exception class to catch a narrower exception. For example, if you are calling a method that declares `Exception` in its throws clause, you can use `catch(Throwable e)` to handle that exception:

```
public static void message() throws Exception{  
    //some code that throws an exception  
}  
  
public static void main(String[] args) {  
    try{  
        message();  
    }catch(Throwable e){  
        e.printStackTrace();  
    }  
}
```

The reverse, i.e., using a catch block with a narrower exception to catch a broader exception is not acceptable. Thus, the following will not compile:

```
public static void main(String[] args) {  
    try{  
        message();  
    }catch(java.io.IOException e){ //catch clause is too narrow to  
catch Exception  
        e.printStackTrace();  
    }  
}
```

You can think of a catch clause as a basket of a particular size. You can use a bigger basket to catch a smaller exception, but you cannot use a smaller basket to catch a bigger exception. Note that I am using the word "bigger" here in the sense that a bigger exception has a bigger tree of subclasses than a smaller exception. For

example, `Throwable` has a lot more subclasses than `Exception` because the tree of `Throwable` includes the tree of `Exception` as well as the tree of `Error`. Similarly, `Exception` is bigger than `IOException` because the tree of `Exception` includes the tree of `IOException` along with a lot of other exception subclasses.

If a method doesn't want to catch the exception then it must declare that exception (or its superclass) in its `throws` clause. This is no different from the rule that you have seen before while creating a method that throws an exception. For example, assuming that the method `message` throws `Exception`, here are valid throws clauses for a method that invokes `message`:

```
//declare the same exception class
public static void callMessage() throws Exception {
    message();
}

//declare a super class of the exception class thrown by message
public static void callMessage() throws Throwable {
    message();
}
```

Of course, `callMessage` is not limited to throwing just the exceptions declared in the throws clause of `message`. It can add its own exceptions to the throws clause irrespective of whether the code inside the method throws them or not.

To catch or to throw 🙌

The decision to catch an exception or to let it propagate to the caller depends on whether you can resolve the problem that resulted in the exception being thrown or not. Consider the following code for a method that computes simple interest:

```
public static double computeInterest(double p, double r, int t)
throws Exception{
    if(t<0) throw new Exception("t must be > 0");
    else return p*r*t;
}
```

and the following code that uses the above method:

```
public static void main(String[] args){
    double interest = 0.0;
    try{
        computeInterest(100, 0.1, -1);
    }catch(Exception e){
    }
    System.out.println(interest);
}
```

Upon execution, the main method prints interest as `0.0` even though the `computeInterest` method did not really compute interest at all. It threw an exception because `t` was less than 0. However, as a user of the program, you won't know that there was actually a failure during the computation of interest.

While the `computeInterest` method did its job of telling the `main` method of a problem in computation by throwing an exception, the `main` method swept this problem silently under the rug by using an empty `catch` block. This is called "**swallowing the exception**" and is a bad practice.

The purpose of a `catch` block is to resolve the problem and not to cover up the problem. By covering up the problem, the program keeps running but starts producing illogical results. Ideally, `main` should not have caught the exception but declared the exception in its `throws` clause because it is in no position to resolve the problem. It would have been appropriate for a program with GUI to catch the exception and ask the user to input valid arguments.

Sometimes, it becomes necessary to catch an exception even though no resolution is possible at that point. The right approach in such a case is to log the exception to the console so that the program can be easily debugged later by inspecting the logs.

```
public static void main(String[] args){
    double interest = 0.0;
    try{
        computeInterest(100, 0.1, -1);
    }catch(Exception e){
        e.printStackTrace();
        //or System.out.println(e);
    }
    System.out.println(interest);
}
```

15.3.4 Quiz 🙋

Q 1. Identify correct statements about the following code:

```
class AccountingException extends Exception{
    int acctNo;
    AccountingException(int acctNo, String msg){ super(msg); this.acctNo =
acctNo; }
}
class LowBalanceException extends AccountingException{
    LowBalanceException(int acctNo, String msg){ super(acctNo, msg); }
}
public class TestClass {
    static void txfrFunds(int fromAcct, int toAcct, int amt) throws
Exception{
```

```

    try{
        throw new AccountingException(fromAcct, "Can't do!");
    }catch(AccountingException ae){
        throw new LowBalanceException(ae.acctNo, "Low balance!");
    }
}
public static void main(String[] args) throws Exception{
    txfrFunds(1, 2, 100);
}
}

```

Select **3** correct options.

- A.** The `txfrFunds` method must declare `AccountingException` in its `throws` clause.
- B.** The `main` method need not declare any exception in its `throws` clause.
- C.** The `txfrFunds` method must declare either `LowBalanceException` or any of its super classes in its `throws` clause.
- D.** The `main` method must declare either `LowBalanceException` or any of its super classes in its `throws` clause.
- E.** The `main` method must declare either the exception or the super class of the exception declared in the throws clause of the `txfrFunds` method.
- F.** As given, the program will compile fine and will end with a `LowBalanceException` when run.
- G.** The program will end with whatever exception is declared in the throws clause of the `main` method.

The correct answer is **C, E, F**.

A method needs to declare the class or its ancestor class of a checked exception that it can potentially throw to its caller in its `throws` clause. In this case, the code within the `txfrFunds` method throws an `AccountingException` but this exception is caught by the `catch` clause. Since the code in the `catch` clause throws a `LowBalanceException`, this is the exception that the caller of the method gets. Therefore, the method needs to declare `LowBalanceException` or any of its ancestor class (which includes `Exception`) in its `throws` clause. Hence, option **A** and **B** are incorrect but **C** is correct.

Since the `main` method does not handle any checked exception that it may receive from the `txfrFunds` method, that exception will be propagated to the caller of the `main` method. Therefore, the `main` method too must declare that exception or its super class in its `throws` clause. However, the main method does not know which exact exception will be thrown by the `txfrFunds` method at run time. It only knows what the `txfrFunds` method declares in its `throws` clause. Therefore, it must declare the checked exception (or its ancestor class) declared in the `txfrFunds` method's `throws` clause. Hence, option **D** is incorrect but **E** is correct.

At run time, the `txfrFunds` method ends up throwing a `LowBalanceException` to the `main` method. The `main` method does not handle this exception and so, this exception will be propagated to the JVM. Hence, option **F** is correct.

Q 2. Given the following exception class code:

```
class BadValueException extends RuntimeException{
    BadValueException(String message){
        super(message);
    }
    BadValueException(String message, Throwable cause){
        super(message, cause);
    }
}
```

```
}
```

and the following program:

```
public class TestClass{
    public static void main(String[] args) throws BadValueException{
        String[] sa = {"bob", null, "charlie"};
        String concatenated = "";
        try{
            try{
                for(String s : sa){
                    if(s == null) throw new BadValueException("Bad Value");
                    else concatenated = concatenated + s;
                }
            }catch(BadValueException bve){
                throw new BadValueException("Unacceptable", bve);
            }
        }catch(Exception e){
            System.out.print("Exception caught : "+e.getMessage()+" ");
        }
        System.out.println(concatenated);
    }
}
```

What will be the output when `TestClass` is run?

Select **1** correct option.

- A. Exception caught : Unacceptable bob
- B. Exception caught : Bad Value bob

C. Exception caught : Unacceptable Bad Value bob

D. Exception caught : Unacceptable Bad Value bobcharlie

E. The program will terminate after printing an exception stack trace.

First of all, observe that `BadValueException` is an unchecked exception and so, there is no need to declare it in the `throws` clause of the `main` method. It is not an error to do so, though.

The second iteration of the for loop throws a `BadValueException` with message `Bad Value` and since the `for` loop is wrapped inside a try/catch block that is able to catch `BadValueException`, this exception is caught by that catch block. The exception causes the `for` loop to break and string pointed to by `concatenated` is just `"bob"`. The `catch` block throws a new `BadValueException` with the message `Unacceptable` and the old exception object as its cause. Since this try/catch block itself is also surrounded by a try/catch block that is capable of catching any exception that extends `Exception`, this second exception is caught by the outer try/catch block. The code in this catch block prints `Exception caught` and the message containing in the exception that it caught, which is `Unacceptable`. Note that it does not print anything from the "cause" exception. If you want to print the message in the cause exception, you can do `e.getCause().getMessage()`. Finally the value of the string pointed to `concatenated` is printed.

15.4 Advanced techniques in exception handling

15.4.1 Using multiple catch blocks

The Java standard library has a large number of exception classes. On top of it, Java allows you to create your own exception classes as well. The purpose of having so many exception classes instead of having a few generic exception classes is to capture as many details of an exceptional situation as possible. These details help the caller of

a method resolve the problem. For example, instead of throwing a general `IOException` when a file is not found, you should throw a more specific `FileNotFoundException` because it describes the problem more accurately to the caller. If a method expects to encounter different issues, it should throw a different exception for each issue. The caller can use multiple catch blocks to catch these exceptions and take action based on which exception is thrown by a method. Here is how:

```
import java.io.*;
public class TestClass{

    //assume that CharConversionException, FileNotFoundException,
    and EOFException extend IOException

    void process(int x) throws IOException{
        if(x == 0) throw new CharConversionException();
        else if(x == 1) throw new FileNotFoundException();
        else throw new EOFException();
    }

    public static void main(String[] args){
        TestClass tc = new TestClass();
        try{
            tc.process(2);
        }catch(EOFException eofe){
            System.out.println("End of file reached");
        }
        catch(CharConversionException cce){
            System.out.println("Some problem with file system");
        }
        catch(FileNotFoundException fnfe){
            System.out.println("No such file found");
        }
        catch(IOException ioe){
            System.out.println("Unknown I/O Exception");
        }
    }
}
```



```
    }  
  }  
}
```

In the above code, if the call to `process` throws an exception, the control will check each catch clause one by one to see if it is able to catch the exception. The control will enter the first catch block that is able to handle the exception. So, for example, if the call to `process` throws `FileNotFoundException`, the first catch block that is able to handle this exception is the third one, i.e., the `catch(FileNotFoundException fnfe)` block. The last catch clause, i.e., `catch(IOException ioe)` is also able to handle a `FileNotFoundException` but the control will not bother with it because it has already found a matching catch block before it reaches the `catch(IOException )` clause.

Unreachable catch blocks 🙅

The above program makes for an interesting problem. What if you move the `catch(IOException ioe)` block before the `catch(FileNotFoundException fnfe)` block? Well, the code will fail to compile. The `catch(IOException ioe)` clause will always satisfy a `FileNotFoundException` and so the control will never get to enter `catch(FileNotFoundException fnfe)` block. Therefore, the compiler will deem the `catch(FileNotFoundException fnfe)` block as unreachable. It is like putting a bigger basket above a smaller one. The smaller basket will never be able to catch anything because of the bigger one sitting above it!

A similar issue will occur if you change `process`'s throws clause in the above program from `throws IOException` to `throws CharConversionException, FileNotFoundException, EOFException`. In this case, the compiler will realize that `process` cannot throw an `IOException` (because it is not listed in `process`'s `throws` clause anymore) and all three exceptions that it can actually throw are being caught already by the three catch blocks and therefore, the `catch(IOException ioe)` block will never be executed. Ideally, the compiler should refuse to compile the code but the JLS allows this and only requires a compiler to generate a warning.

Wait, there is more. What if you change the `catch(IOException ioe)` clause in the `main` method to `catch(Exception e)`. You know that `process` doesn't throw `Exception`. So, what do you think the compiler will do? Well, actually, the compiler won't have an issue with it. Remember that one of the subclasses of `Exception` is `RuntimeException`, which is an unchecked exception. Any method is free to throw a `RuntimeException` without declaring it in its throws clause. Therefore, the compiler needs to consider the possibility that `process` may throw a `RuntimeException` and the `catch(Exception e)` block will get executed if that happens. Thus, the compiler has no choice but to accept the code.

Nested try statements 🙌

It is possible to nest `try` statements inside other `try`, `catch`, or `finally` blocks. The rules for such `try` statements are exactly the same as those for regular `try` statements. But remember that a `catch` block associated with a nested `try` statement cannot catch an exception thrown by the outer `try` block. Here is an example:

```
void process() throws EOFException {
    try{
        if(true) throw new FileNotFoundException();
        else throw new EOFException();
    }catch(FileNotFoundException fnfe){
        try{
            throw new EOFException();
        }catch(EOFException eofe){
            eofe.printStackTrace();
        }
    }
}
```

In the above code, the `catch(EOFException eofe)` block is associated with the `try` statement that is nested inside the `catch(FileNotFoundException fnfe)` block. Therefore, it cannot catch an `EOFException` thrown from the outer `try` block. In fact, if the outer `try` block throws `EOFException`, the control will not even

enter the `catch(FileNotFoundException )` block because `catch(FileNotFoundException fnfe)` clause does not satisfy `EOFException`. Thus, the `EOFException` thrown from the outer `try` block will remain unhandled, which means it must be declared in the `throws` clause of the method.

Throwing an exception from catch or finally 🙌

It is possible to rethrow the same exception (or throw a new one) from the catch block. You may want to do this if you want to do something upon receiving an exception before letting it propagate up the call chain. Of course, if it is a checked exception you would have to declare it in the throws clause of the method. Here is an example:

```
void doIt() throws EOFException {
    try{
        process();
    }catch(EOFException eofe){
        //do something here and then rethrow the same exception
        again.
        throw eofe;
    }
}
```

The same effect can be achieved with a `finally` block:

```
void doIt() throws EOFException {
    try{
        process();
    }finally{
        //do something here
    }
}
```

Since there is no catch block to catch the `EOFException` thrown by `process`, the exception will automatically be thrown out of the `doIt()` method to the caller of `doIt()`, but only after the code in the `finally` block is executed.

Rethrowing an exception explicitly from a `catch` block is helpful only if you have multiple `catch` blocks and you want to do a different thing in every `catch` block.

It is also possible to throw an entirely new exception from the catch block. Here is an example:

```
void payInstallment(double amount) throws Exception {
    try{
        if(this.balance ≥ amount) reduceBalanceBy(amount);
        else throw new LowBalanceException();
    }catch(LowBalanceException lbe){
        throw new Exception("Insufficient Balance", lbe); //passing lbe as
        the cause
    }
}
```

The `LowBalanceException` thrown by `reduceBalanceBy` will be caught by the `catch(LowBalanceException eofe)` block, which then creates an `Exception` object with an informational message and the original exception as the cause, and throws that exception object. Since the type of the exception thrown by the catch block is `Exception`, `payInstallment` must list `Exception` (or its super class) in its `throws` clause.

15.4.2 Using multi-catch 🙌

As you saw in the previous section, having multiple catch blocks for a try block is helpful when you want to do different things upon receiving different types of

exceptions. But what if the code in the try block throws different types of exceptions and you want to do the same thing, say, log the exception message, for each of those types of exceptions? That is easy, actually. You just put a catch clause with the most specific superclass of all those exceptions. For example, if the code throws `FileNotFoundException`, `ArrayIndexOutOfBoundsException`, `NullPointerException`, and `IllegalArgumentException`, and if you want to treat all of them the same way, you could have the following catch block:

```
try{
    // code that may throw any one of FileNotFoundException,
    // ArrayIndexOutOfBoundsException, NullPointerException, and
    // IllegalArgumentException
}
catch(Exception e) { //catch a common superclass exception
    System.out.println(e.getMessage()); //all the exceptions will get
    //logged here
}
```

Let us make it a little more interesting. What if you want to log only `FileNotFoundException` and `ArrayIndexOutOfBoundsException` and re-throw all other `RuntimeException`s? This is not that easy because if you put a catch clause that uses a super class of `FileNotFoundException` and `ArrayIndexOutOfBoundsException` (which is `Exception`, by the way), it will catch all the four exception classes because `Exception` is a superclass of all them. If you put a catch clause that uses a super class of `NullPointerException` and `IllegalArgumentException` (which is `RuntimeException`, by the way), it will catch `ArrayIndexOutOfBoundsException` as well. So, you will need at least three catch blocks.

Java designers introduced a new way to solve this problem in Java 7 in the form of a **multi-catch** block. Take a look at the following code:

```

try{
    // code that may throw any one of FileNotFoundException,
    // ArrayIndexOutOfBoundsException, NullPointerException, and
    // IllegalArgumentException
    if(true) throw new FileNotFoundException();//an easy way to
    simulate the possibility of FNFE
}
catch(FileNotFoundException | ArrayIndexOutOfBoundsException e) { //catches
only FileNotFoundException and ArrayIndexOutOfBoundsException
here
    System.out.println(e.getMessage()); //only the above two exceptions
will get logged here
}
catch(RuntimeException re) { //catches all other RuntimeExceptions here
    throw re;
}

```

The `catch(FileNotFoundException | ArrayIndexOutOfBoundsException e)` clause is called a multi-catch clause because it allows you to catch multiple exceptions in the same catch clause. Although this feature is not used very often in real life, you may see a question or two on it in the exam. So, let us check out the rules for this clause. There are only two of them really.

1. Any number of classes may be listed in a multi-catch clause but none of them may have an ancestor/successor relationship between them. For example, you cannot have `catch(NullPointerException | Exception e)` because `Exception` is an ancestor class (a grandparent, actually) of `NullPointerException`. This restriction makes sense because if a catch clause can catch `Exception`, it will catch subclasses of `Exception` as well. Therefore, adding `NullPointerException`, or any other subclass of `Exception`, to that catch clause would be redundant. Note that the order of

1. exception classes in a multi-catch clause does not matter.
2. The declared type of the catch variable is the same as the most specific superclass of the exceptions listed in the multi-catch clause. For example, the type of the variable `e` in `catch(NullPointerException | IllegalArgumentException e)` will be `RuntimeException`. Of course, at run time, if an exception is thrown in the try block, `e` will point to whichever exception object was thrown.

15.4.3 The try-with-resources statement 📌

A typical usage of a try/catch block is to open and use a resource (such as a file, a network socket, a database connection, or a handle to a remote object) in the `try` block, perform exception handling in the `catch` block, and perform clean up in the `finally` block. Mostly, the clean up involves closing the resources that were opened or used in the `try` block. For example:

```
FileInputStream fis = null; //declared outside the try block so that  
fis will be in scope in the finally block  
try{  
    fis = new FileInputStream("test.txt"); //instantiated inside the  
try block because it may throw an exception  
    byte[] data = fis.readAllBytes();  
    System.out.println(new String(data));  
}catch(IOException e){  
    System.out.println("Exception while opening or reading the file  
"+e.getMessage());  
}  
finally{  
    //close the input stream if it is open  
    if(fis != null){ //it might be null if the open operation
```

```

failed
    try{
        fis.close();//need to wrap it in a try/catch block because even
the close method declares IOException
    } catch(IOException ioe){
        //tried our best to close the resource
        //don't care if the close operation fails
    }
}
}

```

There are two issues with the above code. One, it is tedious. All we want to do is, "close the resource if it is open" but the lines of code required to do this are too many. Furthermore, they are repeated wherever the code uses any kind of resource. Second, it is very easy for the programmer to forget to close the resource after use, which may cause a resource leak.

Java designers came up with a solution to this problem in the form of a **try-with-resources** statement. Take a look at the code below:

```

try(FileInputStream fis = new FileInputStream("test.txt") ) {
//observe this line of code
    byte[] data = fis.readAllBytes();
    System.out.println(new String(data));
}catch(IOException e){
    System.out.println("Exception while opening or reading file "+e);
}

```

Instead of just `try`, we now have `try(FileInputStream fis = new FileInputStream("test.txt") )`. The bracketed part is where you specify the resources that you are planning to use in the upcoming `try` block. The Java compiler automatically adds code to close the resources that you specify in the brackets after the last line of code in the `try` block. I will get into the details of how the JVM closes the resources shortly but for now you can imagine that it is like wrapping the code inside the try-with-resources block within another hidden try/finally block.

Something similar to this:

```
try(FileInputStream fis = new FileInputStream("test.txt") ) {  
    try{ //implicit try block  
        byte[] data = fis.readAllBytes();  
        System.out.println(new String(data));  
    }finally{ //implicit finally block  
        //code that closes the resources specified in try-with-  
resources clause is added by the compiler here  
    }  
}catch(IOException e){  
    System.out.println("Exception while opening or reading file "+e);  
}
```

It is important to understand that since the implicit try/finally block is nested inside the original try-with-resources block, the code to close the resources is executed as a part of the original try block and if that code throws any exception, that exception can also be caught by the original catch block. If you are not interested in handling any exception that may be thrown from the `try` block, you can remove the `catch` clause as well. This simplifies the above code even further:

```
try(FileInputStream fis = new FileInputStream("test.txt") ) {  
    byte[] data = fis.readAllBytes();  
    System.out.println(new String(data));  
}
```

Of course, now you will have to declare `IOException` in the throws clause of the method in which the above code exists. As you can see, this feature makes the code robust as well as easy to read. The resources that you specify in the try-with-resources clause are also called "protected" resources. Since the closing of the resource is handled automatically, this feature is also known as "automatic resource management" .

Syntax of try-with-resources 🙌

The new thing in the try-with-resources statement is that you can specify parameters to the `try` statement like you do for a method. However, unlike method parameters, these parameters are separated with a semi-colon(`;`) instead of a comma(`,`). Furthermore, it allows you to specify variable declarations as well as existing reference variables. Here are a few valid examples:

```
FileInputStream fis1 = new FileInputStream("test1.txt");

try(fis1;) { } //using a single predefined, preinitialized variable
               (the semi-colon is optional)

try(FileInputStream fis2 = new FileInputStream("test2.txt"); fis2;){
} //declaring a new variable as well as using a predefined
  reference variable (the last semi-colon is optional)

try(
    Connection db = connectionManager.getConnection();
    var ss = new ServerSocket(8888); //using a var declaration
    FileInputStream fis2 = new FileInputStream("test2.txt");
    fis1
){ //multiple declaration statements are allowed
}
```

There is no restriction on the type or the number of resources that you want protected. However, the variables that you specify here must be **final** or **effectively final**.

Furthermore, the scope of the variables defined in the try-with-resources clause is the `try` block. In other words, you cannot access these variables in the `catch` block or `after` the try statement. Here are a few invalid usages:

```

FileInputStream fis1 = null;
try(fis1 = new FileInputStream("test1.txt")) //will not compile. This
is neither a variable declaration nor a reference variable.
{ }

try(
    FileInputStream fis1 = new FileInputStream("test1.txt"), fis2 =
new FileInputStream("test2.txt");
    ){ //can't declare multiple variables in the same declaration
statement.
}

try(FileInputStream fis = new FileInputStream("test1.txt")){
    //do something with fis
}finally{
    fis.reset(); //will not compile because fis is out of scope
}

```

Closing the resources 🙋

Closing a resource means giving an opportunity to the resource object to release any system resources that it might have acquired. It is important to release system resources if they are not required so that they can be made available to other users. Conventionally, classes that acquire resources provide a `close()` method, which one can invoke to signify that the instance is not needed anymore. The `close()` method contains code to release the resources acquired by that object.

Java 7 formalized this convention into an interface named `java.lang.AutoCloseable`. This interface declares just one method: `public void close() throws Exception`. The try-with-resources statement builds upon this interface by allowing only those classes that implement this interface to be used in the try-with-resources clause. If a class implements `AutoCloseable`, the compiler is sure that the `close()` method can be invoked on its instances. This

allows the compiler to generate appropriate calls for releasing the resources. Java 7 also retrofitted several preexisting classes to implement this interface. Most significantly, a lot of file I/O classes already implemented the `java.io.Closeable` interface even before Java 7. Java 7 made `java.io.Closeable` extend `java.lang.AutoCloseable`, thereby making all those classes amenable to automatic resource management in one fell swoop. Observe that `AutoCloseable` is in `java.lang` package because it is not specific to I/O resources. In other words, as far as Java is concerned, any class that implements `AutoCloseable` is a resource that can be managed automatically.

Order of closing the resources 🙌

The `close()` method is invoked on each of the resources specified in the try clause in reverse order of their declaration in the try-with-resources clause. The following code shows a dummy resource class named `Device` containing print statements to help us understand this process:

```
public class Device implements AutoCloseable{
    private int index;
    public Device(int index){
        this.index = index;
        System.out.println("Device "+index+" opened");
    }

    public String read() throws Exception{
        if(index == 2) throw new Exception("Unable to read Device "+index);
        System.out.println("Device "+index+" reading");
        return "Device "+index+" data";
    }

    public void close(){
        System.out.println("Device "+index+" closed");
    }
}
```

```
}  
}
```

The following class tests the use of `Device` :

```
public class TestClass{  
    public static void main(String[] args){ //don't need throws  
clause here because of the catch block  
        Device d1 = new Device(1);  
        try(d1; Device d2 = new Device(2)){  
            d1.read();  
            d2.read(); //throws Exception  
        }catch(Exception e){  
            System.out.println("Exception received : "+e);  
        }finally{  
            System.out.println("In Finally");  
        }  
    }  
}
```

The following output generated when `TestClass` is run:

```
Device 1 opened  
Device 2 opened  
Device 1 reading  
Device 2 closed  
Device 1 closed  
Exception received : java.lang.Exception: Unable to read Device 2  
In Finally
```

You should observe the following points:

1. Resources are opened in the **order** specified in the `try` clause (Device 1 and

1. then Device 2).
2. The `close` method is called on the resources in **reverse order** (Device 2 and then Device 1).
3. The `close` method is called on both the resources mentioned in the try clause, even though only Device 2 was instantiated in the try clause. Device 1 was instantiated earlier but was included in the try-with-resources clause.
4. The `catch` and `finally` blocks are executed **after** the resources are closed.

You should try adding a `throw` statement in the constructors and/or the `close` method of `Device` and then observe the output.

Managing exceptions encountered while closing resources 🙌

The exact mechanism of resource closure is a bit more complicated than just wrapping the code in a try/finally block. Specifically, this simplification does not explain what happens if the close method of one resource throws an exception. Are the rest of the resources still closed? What if more than one resource throws an exception while closing? What if the code in the try block also throws an exception - which exception does the caller receive? Although the **OCP 17** exam does not expect you to know these details, the **OCP 21** exam does.

Along with the introduction of the try-with-resources statement, Java 7 also added the concept of "**suppressed exceptions**", which is utilized to address the issues that I just mentioned. Suppressed exceptions is an array of Throwables that can be used to store multiple `Throwable` objects within a `Throwable` object. The benefit of maintaining this array is that you can perform multiple tasks in a method and also remember all the exceptions that are thrown while performing those tasks before throwing that single exception from the method. The caller of the method can look at the array of suppressed exceptions and know of all the exceptions that were generated during the execution of the method. The following example illustrates what I mean:

```
public class SuppressedTest {  
    public static void doSmallTask1(){
```

```

    throw new RuntimeException("Exception in small task 1");
}
public static void doSmallTask2(){
    throw new RuntimeException("Exception in small task 2");
}
public static void doBigTask() throws Exception{
    Exception finalException = new Exception("Exception while
processing big task");
    try{
        doSmallTask1();
    }catch(Exception e){
        finalException.addSuppressed(e);
    }
    try{
        doSmallTask2();
    }catch(Exception e){
        finalException.addSuppressed(e);
    }
    if(finalException.getSuppressed().length>0){
        throw finalException;
    }
}

public static void main(String[] args) {
    try{
        doBigTask();
    }catch(Exception e){
        e.printStackTrace();
        Throwable[] suppressed = e.getSuppressed();
        for(Throwable t : suppressed){
            System.out.println("Suppressed Exception "+t.getMessage());
        }
    }
}

```

```
}  
  
}
```

The above code generates the following output:

```
java.lang.Exception: Exception while processing big task  
  at SuppressedTest.doBigTask(SuppressedTest.java:9)  
  at SuppressedTest.main(SuppressedTest.java:27)  
Suppressed: java.lang.RuntimeException: Exception in small task 1  
  at SuppressedTest.doSmallTask1(SuppressedTest.java:3)  
  at SuppressedTest.doBigTask(SuppressedTest.java:11)  
  ... 1 more  
Suppressed: java.lang.RuntimeException: Exception in small task 2  
  at SuppressedTest.doSmallTask2(SuppressedTest.java:6)  
  at SuppressedTest.doBigTask(SuppressedTest.java:16)  
  ... 1 more  
Suppressed Exception Exception in small task 1  
Suppressed Exception Exception in small task 2
```

Observe that `doBigTask` throws just one exception but that exception contains two suppressed exceptions. Each of the suppressed exceptions contains the stack trace of that particular exception.

So, how is this relevant to the try-with-resources statement? Well, this is the mechanism that the JVM uses to manage exceptions thrown by the code inside the try-with-resources block and by the `close()` methods of the managed resources. Specifically, you need to be aware of two situations:

Situation 1: An exception is thrown by the code in the try block and an exception is thrown by the close method of one or more of the managed resources. In this case, the exceptions thrown by the resource closure are suppressed but are available in the

array of suppressed exceptions of exception thrown by the code in the try block. Let's see how that works by modifying the `close()` method of the above mentioned `Device` class as follows:

```
public void close(){  
    throw new RuntimeException("Unable to close Device "+index+"!");  
}
```

Let us also add `e.printStackTrace();` in the `catch` clause of the `main` method. It now produces the following output:

```
Device 1 opened  
Device 2 opened  
Device 1 reading  
Exception received : Unable to read Device 2  
java.lang.Exception: Unable to read Device 2  
    at exceptions.Device.read(Device.java:10)  
    at exceptions.Device.main(Device.java:23)  
Suppressed: java.lang.RuntimeException: Unable to close Device 2!  
    at exceptions.Device.close(Device.java:16)  
    at exceptions.Device.main(Device.java:21)  
Suppressed: java.lang.RuntimeException: Unable to close Device 1!  
    at exceptions.Device.close(Device.java:16)  
    at exceptions.Device.main(Device.java:21)  
In Finally
```

As you can see, the main method still receives the same exception with the message `"Unable to read Device 2"`, which was thrown from the try block. But this exception now contains two suppressed exceptions that were thrown by the close methods of the two `Device` objects that the JVM attempted to close. The output also proves that the JVM attempts to close every resource mentioned in the try-with-

resource clause (in the reverse order of their declaration in the try-with-resources clause) even if one or more of them throw an exception while closing.

Situation 2: The code in the try block executes without any exception but the close methods of one or more managed resources throw an exception. In this situation, the first exception thrown is the exception received by the caller but that exception contains all of the exceptions encountered while closing the rest of the resources. If you comment out the throw statement from Device's read method, the same program will generate the following output:

```
Device 1 opened
Device 2 opened
Device 1 reading
Device 2 reading
Exception received : Unable to close Device 2!
java.lang.RuntimeException: Unable to close Device 2!
    at exceptions.Device.close(Device.java:17)
    at exceptions.Device.main(Device.java:25)
    Suppressed: java.lang.RuntimeException: Unable to close Device 1!
        at exceptions.Device.close(Device.java:17)
        at exceptions.Device.main(Device.java:22)

In Finally
```

Observe that **Device 2** is closed **first** (because it was **listed last** in the try-with-resources clause) and the exception thrown by its close method is the exception received by the main method. This exception contains the exception thrown by the close method of **Device 1** in its array of suppressed exceptions.

15.4.4 Quiz 📖

Q 1. Which of the following classes can be used with a try-with-resources statement?

Select **1** correct option.

A.

```
class Box{  
    public void open(){ }  
    public void close(){ }  
}
```

B.

```
class Box implements AutoCloseable{  
    public void close(){ }  
}
```

C.

```
class Box implements AutoCloseable{  
    public void open(){ }  
}
```

D.

```
class Box {  
    public final void close(){ }  
}
```

E.

```
class Box extends Closeable{  
    public final void close(){ }  
}
```

The correct answer is **B**.

A try-with-resource statement can only manage an instance of a class that implements `AutoCloseable`. It has exactly one method: `public void close() throws Exception`. Thus, options **A** and **D** are incorrect. Option **C** is also incorrect because

it does not implement the `close` method.

Closeable is actually a pre-Java 7 interface in the `java.io` package and has the same `close` method. It too was meant to represent a resource that can be closed. However, in Java 7 it was made to extend `AutoCloseable` so that all classes that implemented `Closeable` could also be managed using a try-with-resource statement without requiring any change. Thus, option **E** would have been valid if it had said "implements" instead of "extends".

15.5 Exercise

1. Create a method named `countVowels` that takes an array of characters as input and returns the number of vowels in the array. Furthermore, the method should throw a checked exception if the array contains the letter `'x'`.
2. Invoke the `countVowels` method from main in a loop and print its return value for each command line argument. Observe what happens in the following situations: there is no command line argument, there are multiple arguments, there are multiple arguments but the first argument contains an `'x'`. (Use String's `toCharArray` method to get an array of characters from the string.)
3. Ensure that your main method prints the number of vowels in other command line arguments even if one argument contains an `'x'`.
4. Pass null to the `countVowels` method and observe the output.
5. Modify `countVowels` method to throw an `IllegalArgumentException` if it is passed a `null`.
6. Modify `countVowels` method to return `-1`, if the input array is `null` and `0`, if the input array length is less than `10`. Do not use an `if` statement.

7. Create a method `int sum(int[] values, int start, int end)` that throws an `IllegalArgumentException` when passed an array of length 0, a `NullPointerException` when passed a `null`, and `ArrayIndexOutOfBoundsException` when `start` and `end` do not fall within the range of the given array. It should return the sum of the values in the array from `start` to `end` but must throw `Exception` when sum is 0.
8. Invoke the `sum` method created above from `main`. Wrap the call in an appropriate `try/catch` statement. Print two different messages on console depending upon whether the call results in a checked exception or an unchecked exception. Propagate all exceptions up the call chain.
9. Modify the `Device` class introduced in this chapter to throw an exception from its `close` method. Use multiple `Device` objects in a `try-with-resources` block and observe the order of their closure. Also print all the exceptions generated due to the closure of those objects.